Contents

Multi-Tenancy — Patterns, Enforcement, and Codebase Lessons

Last updated: 2026-08-14

What this note is for: a single reference on multi-tenancy — the general concepts (for interviews and for designing a new system from scratch) **and** the lessons from the production platform I work on. It is in two parts:

- **Part I — Patterns.** The five isolation models, shared-schema data modelling, **authentication and authorization** (§I.5.1–I.5.2), the enforcement layers, where leaks happen, operations, testing, and prepared interview answers. Concept-first, system-agnostic.
- **Part II — Codebase lessons.** A self-review of a production multi-tenant B2B SaaS (Turborepo + pnpm monorepo; Express + Prisma + CockroachDB backend, Next.js frontend, feature-flagged modules, Firebase auth). What I built, the mistakes, the tradeoffs I can name and defend, and the improvement roadmap. Client/employer names are scrubbed.

Companion note: eng-labs-platform is the source of truth for *what the system is* — architecture, diagrams, tech stack, current figures. The actionable file-level findings live in the project repo's own

review doc (`documentation_personal/api-development-quality-report.md`), kept out of this personal note deliberately.

Proficiency scale (from this KB): **Aware → Practiced → Proficient → Expert.**

Part I — Patterns: Isolation Models, Enforcement, and Failure Modes

I.0 The framing that makes everything else simple

Multi-tenancy = many customers (tenants) share one system. Every design question reduces to two:

> **Where does tenant data live?** → the *isolation model* **What stops a query crossing the boundary?** → the *enforcement layer*

These are **two independent axes**, and conflating them is the most common mistake. You pick one from each:

	Options
Isolation model (§I.1)	silo · database-per-tenant · schema-per-tenant · pooled · hybrid
Enforcement layer (§I.3)	application code · ORM/repository · database RLS · separate connection/instance

A pooled model with RLS enforcement is far safer than a pooled model with hand-written filters — same data layout, completely different risk profile. Say both when you answer.

Tenant vs user vs organisation. A *tenant* is the unit of isolation and usually the unit of billing — the customer. Users belong to a tenant. Anything that must never be visible across the boundary defines what the tenant is.

I.1 The five isolation models

The comparison table (the thing to memorise)

	1 Silo	2 DB per tenant	3 Schema per tenant	4 Pooled	5 Hybrid
Isolation strength	strongest	strong	medium	weakest	per-tier
Cost per tenant	highest	high	medium	lowest	mixed
Density (tenants/host)	1	tens	hundreds	unlimited	mixed
Noisy-neighbour risk	none	low	medium	high	per-tier

	1 Silo	2 DB per tenant	3 Schema per tenant	4 Pooled	5 Hybrid
Per-tenant schema variation	easy	easy	easy	hard	tiered
Blast radius of a code bug	1 tenant	1 tenant	1 tenant	**all tenants**	tiered
Compliance / data residency	trivial	easy	awkward	hard	easy for the tier that needs it
Migration effort	N deploys	N databases	N schemas	**1**	1 + N
Onboarding a tenant	slow (minutes–hours)	medium	fast	**instant**	mixed
Per-tenant delete / export (GDPR)	trivial	trivial	easy	**hard**	mixed
Cross-tenant analytics	very hard	hard	medium	**trivial**	medium
Ops complexity	highest	high	medium	lowest	highest

Notice the table is almost perfectly **inverted** between isolation and efficiency. That's the whole subject: there is no free lunch, only a choice about which pain you accept.

1 · Silo — full instance per tenant

Separate app deployment, separate database, often separate cloud account/VPC.

- **Use when:** enterprise contracts demand it, data must stay in a specific region, or a regulator requires physical separation. Also the honest answer for a handful of very large customers.
- **Cost:** you now operate N systems. Every release is N deploys; a hotfix is N rollouts; version skew across tenants becomes real.
- **Watch:** this is usually where "we'll just spin up another instance" quietly becomes an ops team.

2 · Database per tenant — shared application

One application fleet; the app resolves the tenant to its own database connection.

- **Use when:** you need strong data isolation and easy per-tenant backup/restore/export, but want one codebase.
- **Cost:** connection-pool pressure (N pools), and migrations must run N times — with partial-failure handling. Restoring one tenant is trivial, which is a genuinely underrated benefit.
- **Watch:** connection limits are the practical ceiling. Hundreds of tenants is fine; tens of thousands is not.

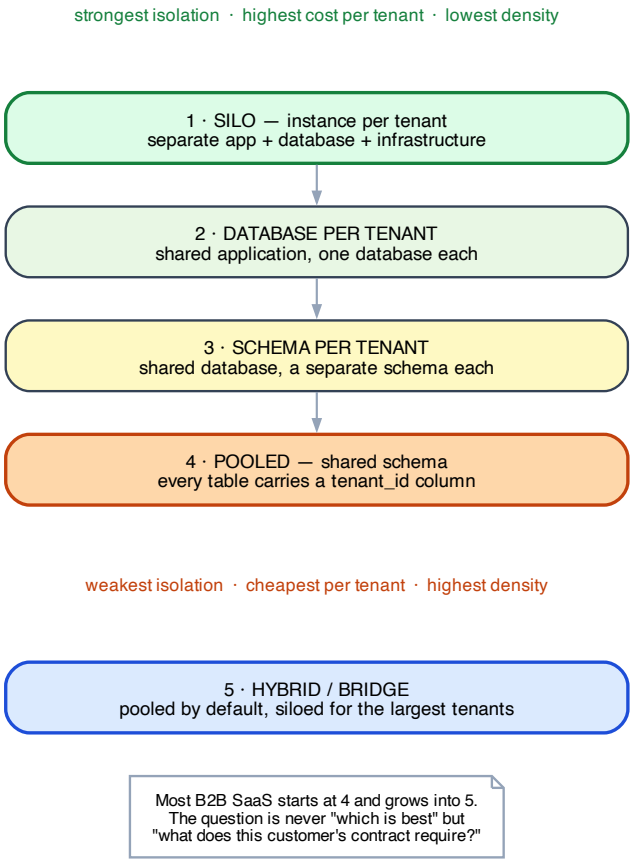

Figure 1: The five multi-tenancy isolation models

3 · Schema per tenant — shared database

One database, one Postgres schema (namespace) per tenant. The app sets `search_path` per request.

- **Use when:** a middle ground — better isolation than pooled, cheaper than a database each.
- **Cost:** the awkward one in practice. Migrations still run N times, catalog bloat becomes a real problem past a few thousand schemas, and many tools and ORMs handle it badly.
- **Watch:** often chosen as a compromise and regretted. Be able to say *why* you'd skip it.

4 · Pooled — shared schema, `tenant_id` discriminator

Every tenant-owned table carries a `tenant_id` column. Every query filters on it. **This is what most B2B SaaS runs, including the platform in Part II.**

- **Use when:** many tenants, fast self-serve onboarding, one migration path, and cross-tenant analytics matter.
- **Cost:** isolation is now a *property of your code*, not of your infrastructure. One missing `WHERE` clause is a data breach. Noisy neighbours share indexes and connections. Per-tenant deletion means finding every table.
- **Non-negotiable mitigations:** enforce at the ORM or database layer (§I.3), never at the call site; index `(tenant_id, …)` as a *leading* column on every hot query; test the cross-tenant negative case (§I.7).

5 · Hybrid / bridge — pooled by default, siloed on demand

Small and mid customers pooled; enterprise customers get their own database or instance. Same codebase, tenant metadata records which tier a tenant is in.

- **Use when:** you've succeeded. This is the natural end state of a growing SaaS — one enterprise contract demands isolation and you don't want to re-architect for everyone.
- **Cost:** you now maintain **both** paths, including two migration strategies and two backup stories. Highest ops complexity of the five.
- **Design implication:** if you think you'll ever get here, **make tenant→datasource resolution an abstraction from day one**, even when it always returns the same connection. Retrofitting that indirection later is the expensive part.

I.2 How to choose — the decision guide

Ask these in order. The first "yes" that carries a contractual or legal obligation wins.

1. **Does any contract or regulation require physical separation or data residency?** → silo or DB-per-tenant for those tenants (which means hybrid).
2. **How many tenants, realistically, in 3 years?** Tens → DB-per-tenant is viable. Thousands+ → pooled.
3. **How is it sold?** Self-serve signup demands instant onboarding → pooled. Enterprise sales with onboarding calls → per-tenant is affordable.
4. **Do tenants need structurally different data models?** Genuinely different → per-tenant schema. Same shape, different labels → pooled with configuration. (See §II.8.2 — flexibility must earn its complexity.)
5. **Is cross-tenant analytics a product feature?** → pooled makes it trivial; everything else makes it a data-warehouse project.

6. **What's the blast radius you can survive?** Pooled means one bad query can affect everyone.

The honest default: start pooled, enforce at the database or ORM layer, and keep tenant→datasource resolution behind an interface so hybrid stays available.

I.2.1 Shared-schema data modelling — the tenant boundary belongs in the data model

For the **shared-schema / pooled model**, one of the most important data-modelling rules is:

> **Every tenant-owned table should generally carry its own `tenant_id`, even when the tenant could theoretically be derived through a foreign-key chain.**

For example:

```
courses

  id   tenant_id


       ↓
course_modules

  id   course_id   tenant_id


       ↓
lessons

  id   module_id    tenant_id
```

Even though `lessons` can theoretically determine its tenant through `module → course → tenant`, storing `lessons.tenant_id` directly gives the database a direct ownership boundary.

This is especially useful for RLS. A straightforward policy can operate directly on the table:

```
CREATE POLICY tenant_isolation ON lessons
USING (tenant_id = current_setting('app.tenant_id')::text);
```

without requiring the RLS policy to traverse relationships to discover the lesson's tenant.

Why duplicate the tenant key when it is derivable?

It is intentional denormalization. The small amount of redundancy buys:

- **simpler tenant-scoped queries**
- **simpler RLS policies**
- **tenant-aware indexes**
- **easier tenant-scoped deletes and exports**
- **simpler auditing and observability**
- **a direct isolation boundary on every tenant-owned table**

The important distinction is that **not every table needs `tenant_id`**. Genuinely global/reference tables such as `countries`, `currencies`, `languages`, or system-wide permission definitions do not belong to a tenant and therefore do not need a tenant discriminator.

`tenant_id` is the foundation, not the whole design

For a shared-schema SaaS, data modelling is not *only* about adding a `tenant_id` column. The broader goal is:

```
Multi-tenant data modelling

   Tenant ownership
       tenant_id on tenant-owned tables

   Relationships
       prevent cross-tenant relationships

   Constraints
       tenant_id often participates in uniqueness rules

   Indexes
       tenant_id is usually part of relevant indexes

   RLS / isolation
       enforce the tenant boundary

   Global vs tenant data
        don't add tenant_id to genuinely global tables
```

For example, if email addresses only need to be unique **within a tenant**, this is usually wrong:

```
UNIQUE(email)
```

and this is the appropriate tenant-scoped constraint:

```
UNIQUE(tenant_id, email)
```

because two different tenants can legitimately have users with the same email address.

The mental model

> **Every piece of tenant-owned data, relationship, uniqueness rule, query, and authorization boundary must respect the tenant boundary.**

`tenant_id` is the foundation that makes that boundary explicit in a pooled/shared-schema model; RLS, constraints, indexes, and application/ORM scoping build on top of it.

I.3 Where isolation is enforced — the axis that actually decides safety

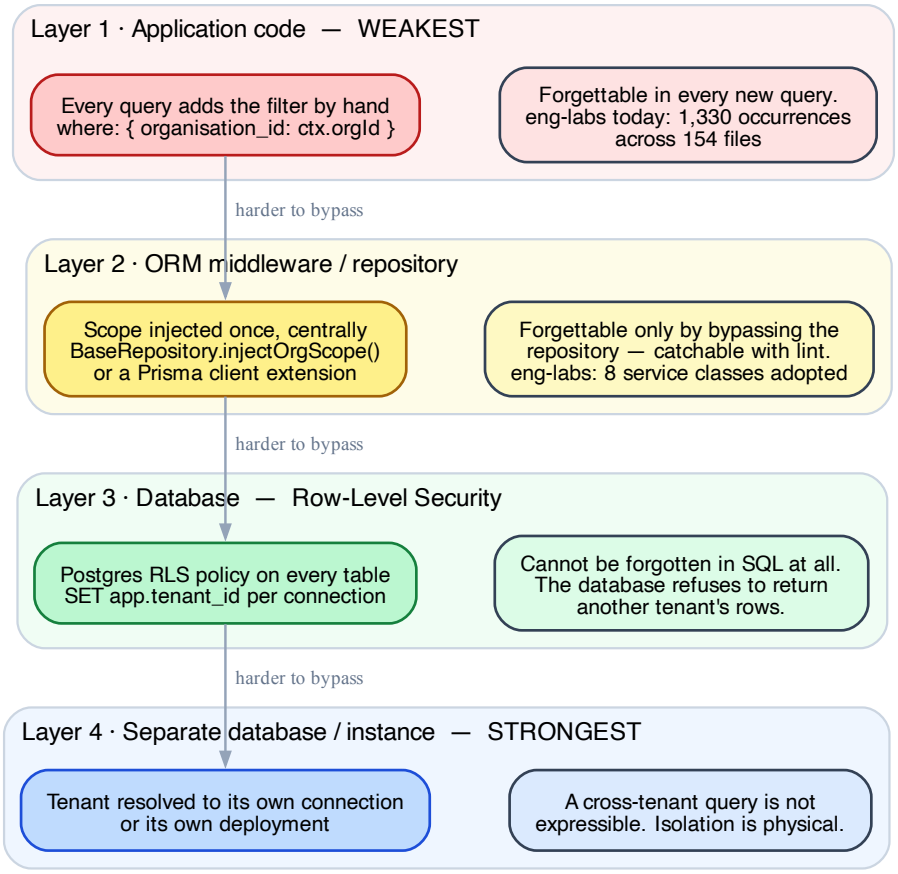

Figure 2: Where tenant isolation can be enforced

Layer	Mechanism	Can a developer forget it?
1 · Application code	every query hand-writes `where: { tenant_id }`	**Yes — in every new query.** No safety net.
2 · ORM / repository	one central scope injector (Prisma client extension, base repository, Django manager, Rails `default_scope`)	Only by bypassing the layer — and lint can catch that.
3 · Database (RLS)	Postgres Row-Level Security policy per table; `SET app.tenant_id` per connection	**No.** The database refuses the rows. Even raw SQL is covered.
4 · Separate connection / instance	tenant resolved to its own datasource	**No.** A cross-tenant query isn't expressible.

Row-Level Security, concretely — worth being able to sketch:

```
ALTER TABLE courses ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON courses
  USING (tenant_id = current_setting('app.tenant_id')::text);

-- then per request / per connection:
SET LOCAL app.tenant_id = 'tnt_abc123';
```

Every subsequent query on that connection is filtered by the database itself. The trade-offs to mention: it needs the setting applied reliably on every connection (easy to get wrong with pooling), it can complicate query plans, and you need a deliberate bypass role for migrations and admin tooling.

The principle underneath all of this: *if a rule must be obeyed everywhere, make it impossible to disobey rather than documenting it.* Layer 1 is documentation. Layers 2–4 are enforcement.

I.4 Where leaks actually happen

Real cross-tenant incidents almost never come from the main CRUD path — that's the code everyone reviews. They come from these six:

Leak path	Why it happens	Fix
Background jobs / queues	there's no request, so no `req.context` to read	serialise `tenant_id` into the job payload; make the worker's data access require it
Reports & aggregates	a `COUNT/SUM/GROUP BY` written directly, filter forgotten	route analytics through the same scoped layer; never hand-write aggregate SQL
Cache keys	key built from entity id without the tenant	make `tenant_id` a mandatory part of every cache key by construction
Admin / impersonation paths	deliberately cross-tenant, then copy-pasted into normal code	keep them in a separate, clearly-named module; audit-log every use

Leak path	Why it happens	Fix
`findUnique` by id alone	ids are globally unique (e.g. nanoid), so the filter feels unnecessary — until an id leaks or is guessed	always scope by tenant *as well as* id; treat id as insufficient authorisation
Raw SQL & migrations	bypass the ORM, so layer-2 enforcement doesn't apply	this is the strongest argument for database-level RLS

> The `findUnique` one is the subtle killer, and the Part II platform hit exactly this: `BaseRepository.findUnique()` is deliberately downgraded to a scoped `findFirst()` because `findUnique` cannot enforce `organisation_id` without changing the unique constraints. Good instinct, and a good interview anecdote.

I.5 "Tenant" touches more than the database

A checklist people forget — the data layer is maybe half the work:

Concern	What it needs
Tenant resolution	how a request declares its tenant: subdomain, path prefix, custom domain, JWT claim, or header. Pick one canonical source; treat the others as untrusted.
Auth & context	tenant on the token/session; validate the user is actually mapped to the tenant they claim (don't trust a header)
Feature gating	which products/modules each tenant has; usually a registry table + a per-tenant enablement table
Config & branding	logo, domain, email templates, locale, timezone
Quotas & rate limits	per-tenant limits, so one tenant can't exhaust shared capacity (the noisy-neighbour mitigation)
Billing & metering	usage counted per tenant, reconciled with the plan
Observability	`tenant_id` on every log line, trace span and metric — otherwise you cannot debug "it's slow for customer X"
File storage	tenant-prefixed paths/buckets; signed URLs scoped to the tenant
Search indexes	tenant as a mandatory filter, or an index per tenant
Data lifecycle	per-tenant export and hard-delete (GDPR); hardest in the pooled model, so design it early

The first two rows carry enough weight to deserve their own sections — §I.5.1 and §I.5.2.

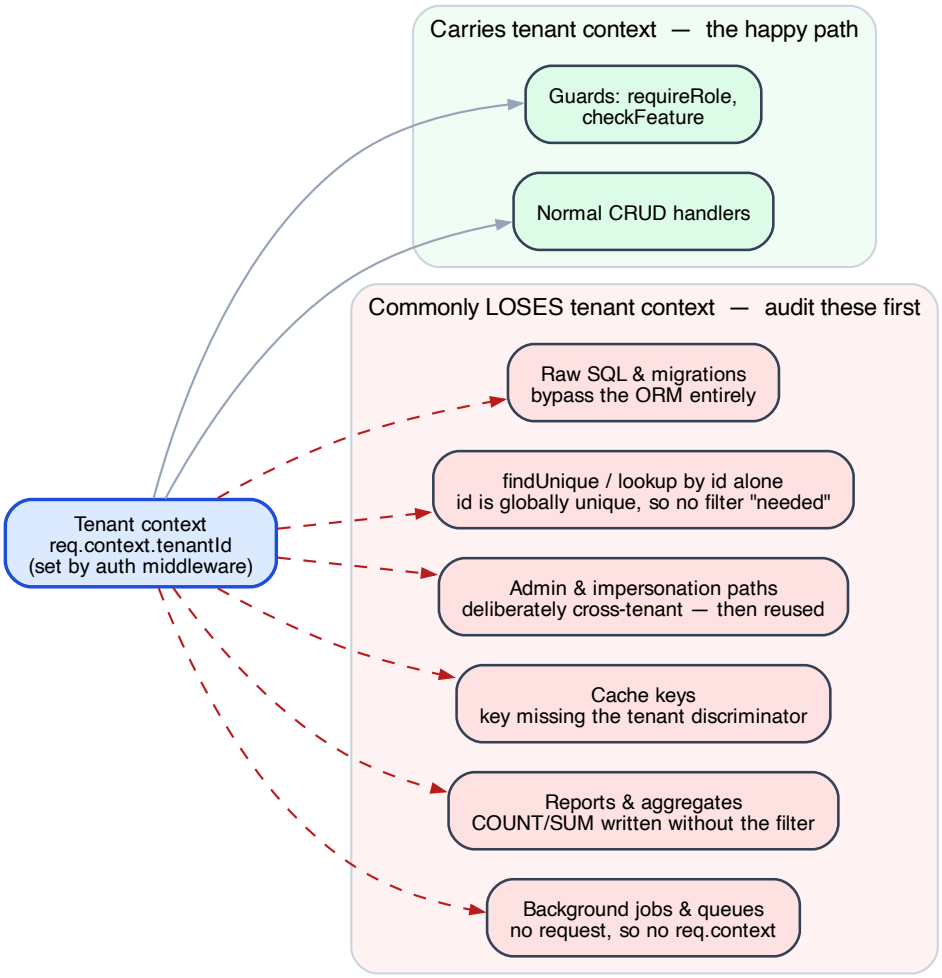

Figure 3: Paths that lose tenant context

I.5.1 Authentication — establishing *who*, and *which tenant*

Authentication answers "who is this request from?" Authorization answers "may they do this?" They are different questions, run in that order, and conflating them is how you get a system where being logged in is mistaken for being allowed.

In a single-tenant app, authentication answers one question. In a multi-tenant system it must answer **three**, and the third is the one people miss:

#	Question	Failure if skipped
1	Who is this user?	impersonation
2	Which tenant is this request for?	queries scope to the wrong tenant, or none
3	**Is this user actually entitled to that tenant?**	**cross-tenant breach — the user simply asks for tenant B**

Question 3 is the multi-tenant-specific one. A request that says "I am Alice, acting in tenant B" must be checked against Alice's actual tenant memberships — every time, server-side.

Session vs token

	Server session	Token (JWT)
State	stored server-side, cookie holds an opaque id	self-contained, signed, no server state
Revocation	**immediate** — delete the session	**hard** — valid until it expires
Scaling	needs shared session store	stateless, scales trivially
Cross-service / cross-domain	awkward	natural
Best for	classic web apps, one domain	APIs, mobile, multiple apps/domains

The trade in one line: **JWTs buy statelessness and pay for it in revocation.** Everything else about token design is managing that debt.

Managing the revocation debt

1. **Short-lived access token + long-lived refresh token.** Access token minutes-to-an-hour; refresh token days, stored server-side so it *can* be revoked. This is the standard answer.
2. **A token version / `token_generation` column on the user.** Bump it on password change or forced logout; reject tokens carrying an older version. Cheap, one indexed read.
3. **A denylist** of revoked jti values until natural expiry. Reintroduces state, so use it only for the genuinely urgent case.

Where does the tenant come from?

Source	Trustworthy?	Notes
Signed token claim	yes	the tenant is inside the signature — tampering invalidates it
Subdomain (`acme.app.com`)	only after mapping	resolve to a tenant id, then verify membership
Path prefix (`/t/acme/...`)	same	readable and shareable; still must be verified
Custom domain	same	needs a domain→tenant table
Plain header / query param	**never alone**	a client can write anything

> **The rule:** pick **one** canonical source, put it in the signed token, and treat every other source as a *request* to switch context — which is validated, not obeyed.

Context switching — how a user with several tenants moves between them

Do **not** let the client pass `?tenant=B`. The correct flow:

```
client asks to switch to tenant B
   → server checks membership: does this user have a mapping to B?
   → yes → mint a NEW token carrying tenantId = B
   → client uses the new token; every later request is self-describing
```

The tenant now travels inside the signature. No downstream handler has to re-validate, and no header can override it.

Cross-app SSO — moving an authenticated user between your own apps

When one product spans several apps/domains, don't share the primary session. Mint a **short-lived, single-purpose bridge token**:

- separate secret from the main auth secret (blast-radius containment)
- very short TTL (minutes) — it exists only to survive one redirect
- **an allow-list of redirect hosts**, enforced server-side, HTTPS only — an open redirect here hands your token to an attacker
- exchanged immediately at the destination for that app's own session/token

Multiple identity providers

Real systems accumulate them: an IdP (Firebase/Auth0/Okta) for humans, self-issued JWTs for service-to-service or for apps you also own. Verify in a defined order and make the fallback explicit. Two rules: **each provider gets its own verification path and its own secret/keys**, and the code must never treat "provider A rejected it" as "the token is invalid" until every provider has been tried.

I.5.2 Authorization — deciding *what they may do*

Pick the coarsest model that expresses your real rules

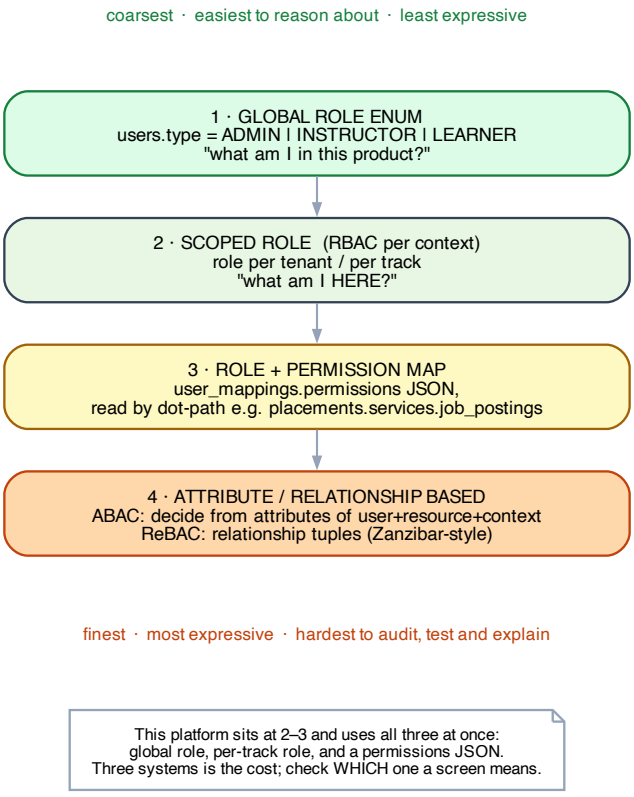

Figure 4: Authorization models, coarse to fine

Level	Model	Good for	Cost
1	**Global role enum** — `user.type = ADMIN`	small products, few rules	can't express per-context differences
2	**Scoped role (RBAC per context)** — role per tenant/project	most B2B SaaS	need to know *which* scope a screen means
3	**Role + permission map** — a JSON/table of granular grants	fine-grained product tiers	untyped, hard to audit, easy to drift
4	**ABAC / ReBAC** — decide from attributes or relationship tuples	sharing graphs, complex hierarchies	needs a policy engine; hardest to test and explain

Each level up adds expressiveness and costs auditability. Most teams need level 2, adopt level 3 for entitlements, and should reach level 4 only when the rules are genuinely relational ("can edit if they're on the team that owns the parent folder").

The three-gate contract

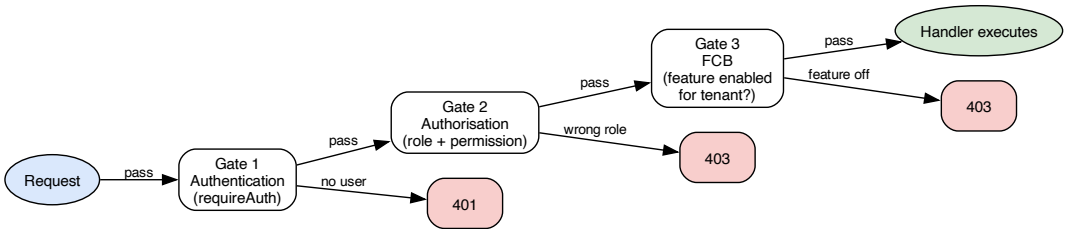

Figure 5: The three-gate request contract

Every protected route passes three gates, composed as middleware, in order:

```
authentication  → is there a valid identity?          401 if not
authorization   → does this identity have the role?   403 if not
entitlement     → has this tenant bought the feature? 403 if not
```

Keeping them separate means each has one reason to fail and one place to fix.

Enforce at one server-side chokepoint

- **The server is the enforcement boundary. The UI is a hint.** Hiding a button while the endpoint stays open is a vulnerability with a polite face.
- **A URL prefix is not protection.** `/admin/*` is a naming convention, not a guard.
- **Fail closed.** Missing context, unknown role, unreadable permission blob → deny. Never "if we can't tell, allow it."
- **Make it structural, then lint it.** A convention that a route "should" have a guard, with nothing failing the build, is a wish (§II.6.2).

Authorization is tenant-scoped too

An easy and dangerous mistake: treating a role as global when it is per-tenant. Being ADMIN of tenant A must confer nothing in tenant B. Every permission lookup must be keyed by `(user, tenant)` — not by user alone.

401 vs 403 vs 404 — and what each leaks

Response	Means	Leaks
401	not authenticated	nothing
403	authenticated, not permitted	**that the resource exists**
404	not found *or* not yours	nothing

For cross-tenant requests, **prefer 404 over 403**. A 403 on tenant B's resource confirms the id is real, which turns id enumeration into a discovery tool. Use 403 only inside a tenant the user can already see.

The failure modes that actually happen

Mode	What it looks like
IDOR	`GET /courses/:id` with no ownership check — a valid id from any tenant works
Mass assignment / privilege escalation	`data: req.body` lets a caller set `role: "SUPER_ADMIN"` on themselves
UI-as-security	button hidden, endpoint open
Prefix-as-security	`/admin/*` assumed protected
Global role in a scoped world	tenant-A admin acts in tenant B
Unguarded new route	the convention was documented, nothing enforced it

The mitigations are dull and effective: allow-list writable fields, scope every lookup by tenant *and* id, compose guards rather than hand-rolling checks, and fail the build for routes without one.

I.6 Operations, per model — what interviews probe

"How do you run a migration across 500 tenant databases?" The expected answer: a migration runner that iterates tenants with per-tenant state tracking, runs in batches, is idempotent and resumable, and tolerates partial failure (tenant 213 failed; the other 499 succeeded and you can retry just that one). Plus a compatibility rule: deploy schema changes that are backward-compatible so app and schema versions can differ mid-rollout.

"How do you restore one tenant's data?" Trivial in models 1–2 (restore that database). In pooled, you need either per-tenant logical exports or point-in-time recovery into a scratch instance followed by a filtered copy. Worth knowing this is a real weakness of pooled.

"How do you onboard a tenant?" Pooled: insert a row. Per-database: provision, migrate, seed, register — a workflow with failure states.

Backward-compatible schema change order (applies to any model, asked constantly): add nullable column → deploy code that writes both → backfill → deploy code that reads new → make non-null / drop old. Never one big-bang migration.

I.7 Testing tenant isolation

The single highest-value test category in a multi-tenant system, and cheap to write:

```
For every endpoint that reads or writes tenant data:
  1. 200  - tenant A's user reads tenant A's resource            (happy path)
  2. 401  - no token                                              (authentication)
  3. 403  - tenant A's user, insufficient role                    (authorisation)
  4. 404/403 - tenant A's user requests tenant B's resource by id (ISOLATION)
```

Cases 1–3 are "triangle coverage." **Case 4 is the one that catches breaches**, and it's the one usually missing. A pooled system without case-4 tests has no evidence its isolation works.

Also worth having: a repository-level test that a query built without tenant context **throws** rather than returning everything. Fail closed, loudly.

The auth cases worth adding

Beyond the four above, these catch the failure modes in §I.5.1–I.5.2 and are equally cheap:

Test	Expect	Catches
Expired token	401	clock/verification bugs
Token signed with the wrong secret	401	accepting unverified claims
Valid token, **tenant claim swapped** to a tenant the user isn't in	401/404	trusting the claim without a membership check
Tenant passed as a **header/query param** while the token says otherwise	token wins	header override — the classic breach
Write endpoint sent an extra `role` / `is_admin` field	field ignored	mass-assignment privilege escalation
Cross-tenant resource by id	**404, not 403**	existence leaking via status code
Route registered with no guard	**CI fails**	the unguarded-new-route mode

That last row is a lint rule, not a test — and it's the only one that prevents the failure rather than detecting it.

I.8 Compact interview answers

"How would you design multi-tenancy for a new B2B SaaS?" > Two decisions, not one: the isolation model and the enforcement layer. I'd default to pooled — shared schema with a tenant discriminator — because it gives instant onboarding, one migration path, and easy cross-tenant analytics.

But pooled makes isolation a property of the code, so I'd enforce it in the data layer, not at call sites: a client extension or Postgres RLS, so a query that forgets the tenant filter is impossible rather than merely discouraged. I'd also keep tenant→datasource resolution behind an interface from day one, so when an enterprise customer demands a dedicated database I can go hybrid without re-architecting.

"What's the biggest risk in the pooled model?" > One missing `WHERE` clause is a data breach, and it won't come from the CRUD path everyone reviews — it comes from background jobs with no request context, hand-written report aggregates, cache keys missing the tenant, or a lookup by a globally-unique id where the filter felt unnecessary. That's why I want enforcement below the application layer, and a test on every endpoint that tenant A gets a 404 for tenant B's resource.

"Tell me about a multi-tenancy mistake you made." > On a platform with 199 models, we enforced tenant scoping in application code — the rule was documented, and every query restated it by hand. It ended up written out in 154 files. Nothing was wrong functionally, but any change to what "scoped" means became a 154-file edit with no compiler help, and each new query was a fresh chance to forget. We'd already written the right abstraction — a base repository that injects the scope once — but it was opt-in, so adoption stalled at 8 service classes. The lesson: if a rule must hold everywhere, make it impossible to break rather than documenting it. The fix is a client extension so it applies without opt-in.

"How do you migrate 500 tenant databases?" > Backward-compatible changes only, so app and schema versions can differ during rollout — add nullable, dual-write, backfill, switch reads, then clean up. And a migration runner that's idempotent, resumable, batched, and tracks state per tenant, so a failure on tenant 213 doesn't block the other 499 or force a full re-run.

"How does authentication work in a multi-tenant system?" > It has to answer three questions, not one: who the user is, which tenant the request is for, and whether that user is actually entitled to that tenant. The third is the one people skip and it's where the breach lives. I'd carry the tenant as a claim inside the signed token rather than a header or query param, so it can't be tampered with and downstream handlers don't have to re-validate. Switching tenants means re-minting the token after a server-side membership check — never obeying a client-supplied tenant id. And because JWTs trade revocation for statelessness, I'd pair a short access token with a server-side refresh token, plus a token-version column so a forced logout invalidates everything immediately.

"How would you design authorization?" > Pick the coarsest model that expresses the real rules — usually scoped RBAC, a role per tenant rather than a global one, because being admin of tenant A must confer nothing in tenant B. Then enforce at one composed chokepoint: authentication, then authorization, then feature entitlement, as three separate gates so each has one reason to fail. The server is the boundary; hidden UI is a hint, not a control. Two details I'd insist on: fail closed when context is missing, and return 404 rather than 403 for another tenant's resource, because a 403 confirms the id is real and turns enumeration into discovery.

"Tell me about a security bug you shipped." > We relied on a URL prefix for protection — `/admin/*` routes were assumed guarded because of the path. The auth middleware populated the request context but never rejected, so protection depended on each route remembering to add a role guard, and most mutating routes hadn't. A LEARNER-role account deleted a course through one of them. Three things came out of it: the middleware authenticates but `requireAuth()` is what rejects, so enforcement is explicit; role guards went from near-zero to widely applied; and the real lesson — a convention nothing enforces is a wish, so the fix isn't documentation, it's failing CI when a route registers without a guard.

I.9 Design checklist for a new multi-tenant system

- ☐ Name the tenant explicitly — what must *never* cross this boundary?
- ☐ How many levels does the hierarchy have? Model each as first-class.
- ☐ Pick the isolation model (§I.2 decision guide) and write down *why*.
- ☐ Pick the enforcement layer — **not** application code.
- ☐ One canonical name for the tenant key; the same name everywhere, including its own table.
- ☐ Tenant→datasource resolution behind an interface, even if it's a constant today.
- ☐ Index `(tenant_id, …)` as a leading column on every hot query path.
- ☐ Tenant context flows into background jobs, cache keys, logs, traces, metrics, storage paths, search.
- ☐ Case-4 isolation test on the first endpoint, and every one after.
- ☐ Repository throws — loudly — if tenant context is missing. Fail closed.
- ☐ Per-tenant export and hard-delete designed before you need them.
- ☐ Per-tenant quotas/rate limits, so no tenant can exhaust shared capacity.
- ☐ Feature entitlement separated from platform kill-switch.

Authentication & authorization (§I.5.1–I.5.2):

- ☐ Tenant travels as a **signed token claim**, never a bare header or query param.
- ☐ Switching tenant re-mints the token **after** a server-side membership check.
- ☐ Short access token + revocable refresh token; a token-version column for forced logout.
- ☐ Each identity provider has its own verification path and its own secret/keys.
- ☐ Cross-app SSO uses a separate short-lived secret **and a redirect host allow-list**.
- ☐ Permission lookups keyed by `(user, tenant)` — no global roles in a scoped world.
- ☐ Three gates composed as middleware: authentication → authorization → entitlement.
- ☐ Fail closed on missing/unreadable context.
- ☐ Writable fields allow-listed; never `data: req.body`.
- ☐ **404, not 403**, for another tenant's resource.
- ☐ CI fails when a route registers without a guard.

Part II — Lessons from the eng-labs codebase (model 4, pooled)

Context: a self-review of the production platform, written after a deep read of the codebase to (a) rate it honestly, (b) name the engineering skills I need to strengthen, and (c) capture the target conventions so a new engineer can onboard easily.

II.0 The system at a glance — and how to re-measure

The hierarchy is four levels, not two

```
Organisation  →  Entity (tenant)  →  Unit  →  Users
```

Most multi-tenancy writing assumes `tenant → user`. Real B2B often needs more: here an Organisation may own several Entities (colleges), each with Units (departments, sections, batches). **Consequence: "scoped" is not one column but a path.** Queries scope by `organisation_id` and typically `tenant_id`, and sometimes need `unit_id` too — which is exactly why the leak surface is large (154 files).

Lesson for a new system: decide how many levels the boundary has *before* the schema, and give each level a first-class model. Retrofitting a level (see the missing cohort/batch level in §II.2.1) is expensive.

Feature gating: two gates, both must pass

```
feature_registry.enabled     ← platform-level kill switch (we control)
        AND
entity_features.enabled      ← tenant-level entitlement (the customer's contract)
```

A clean design worth reusing: it separates "does this product exist" from "has this customer bought it," and lets support disable a broken feature globally without touching entitlements.

Re-measure before using this note

This is a tracking document, so the numbers must be refreshable. **Confirm the branch first** — see §II.6.3, this is not optional.

```
git log --oneline -1 main
git rev-list --left-right --count main...HEAD      # left = main-only, right = HEAD-only

m=main
git show $m:packages/database/prisma/schema.prisma | grep -c '^model '   # models
git ls-tree -r --name-only $m -- apps/api/src/routes | grep -c '\.ts$'   # route files
git grep -l 'prisma\.' $m -- 'apps/api/src/routes/**' | wc -l            # routes bypassing serv
git grep -oh 'requireRole('  $m -- 'apps/api/src/**' | wc -l             # guard adoption
git grep -l  'organisation_id' $m -- 'apps/api/src/**' | wc -l           # scope-leak surface
git ls-tree -r --name-only $m | grep -i legacy                           # unfinished cutovers
```

II.0.5 The auth implementation, end to end

The concrete realisation of §I.5.1–I.5.2 in this codebase. Measured on `main`.

The authentication path

Token source — header beats cookie. The middleware reads `req.cookies.auth_token`, then overrides it with `Authorization: Bearer` if present. The comment in `middleware/auth.ts` explains why: a header is an *explicit client action* and "works better for context switching." The cookie is the ambient session; the header is a deliberate "act as this instead."

Two providers, tried in order (`validateToken` in `packages/helpers`):

1. **Custom backend JWT** — `jwt.verify(token, secret, { algorithms: ["HS256"] })`, secret fetched from **GCP Secret Manager**. Pinning the algorithm matters: omitting it is how `alg: none` attacks work.
2. **Firebase** — falls through to `fAuth.verifyIdToken()`.

On success it loads the user plus `userMappings` (with tenant + features), organisation, and course enrolments. This is §I.5.1's "multiple identity providers" in practice: humans authenticate via Firebase; tokens the platform mints itself (mock-OA, resume bridge) use the backend JWT path.

Context assembly — the precedence rules are the design:

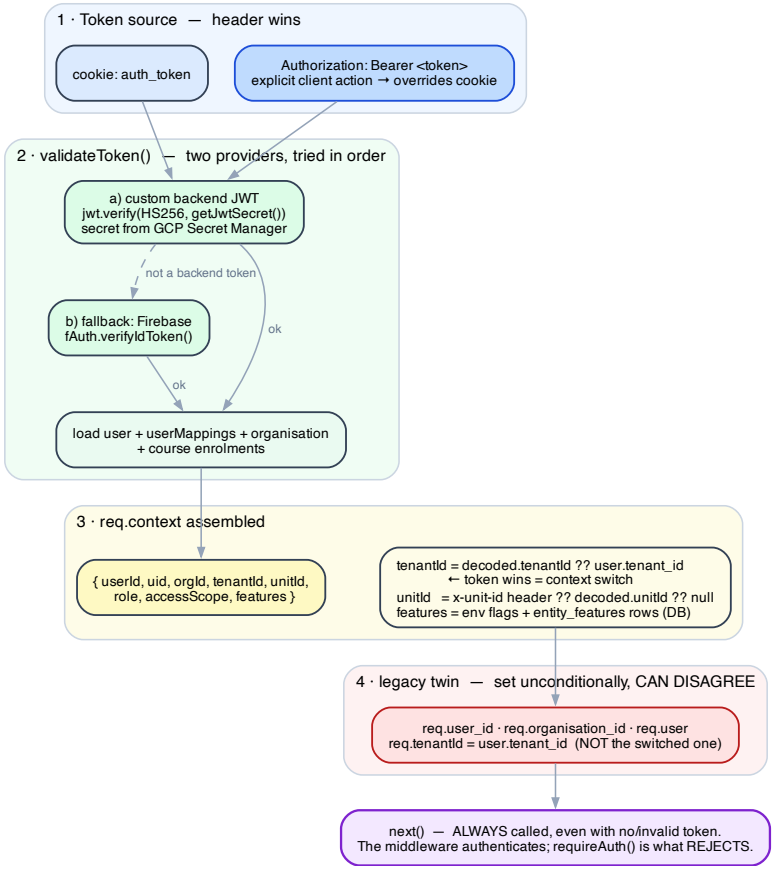

Figure 6: Token sources, dual verification, context assembly

Field	Resolution	Why
`tenantId`	`decoded.tenantId ?? user.tenant_id`	**the token wins — this is how context switching works**
`unitId`	`x-unit-id` header → `decoded.unitId` → `null`	a header is allowed here because a unit is *within* the already-verified tenant
`features`	env flags **merged with** `entity_features` rows for the resolved tenant	so `requireFeature()` works for DB-stored flags, not just env ones
`role`	`user.type`	the global role — see the three-role caveat below

The middleware never rejects. Every path ends in `next()` — no token, bad token, thrown error. It *authenticates*; `requireAuth()` is what returns 401. That separation is deliberate and correct, and it is also exactly what made the May incident possible when routes forgot the guard.

The authorization surface

Guard	Call sites	Checks
`requireAuth()`	296	a valid `req.context` exists
`checkFeature(key)`	162	both feature gates, via DB query
`requireFeature(key)`	47	feature flag already on `req.context.features`
`requireRole(...roles)`	42	role membership
`requireModuleAdminAccess(module, path?)`	38	module admin + nested JSON permission
`requirePlacementPermission(...keys)`	4	granular permission keys

Three role systems coexist — and knowing *which one a screen means* is a real source of bugs:

System	Values	Scope
Platform role (`users.type`)	`ADMIN` · `INSTRUCTOR` · `LEARNER` · `COORDINATOR`	global to the user
Per-track role (`project_enrollments.role`)	`PROJECT_INCHARGE` · `PROJECT_MENTOR` · `LEARNER`	**one per user per track**
Module access (`project_mentor_access.pm_role`)	`PM_ADMIN` · `PM_MEMBER`	per tenant + unit subtree

Level 2 on the §I.5.2 spectrum, with level 3 layered on: `user_mappings.permissions` is a JSON blob read by dot-path — `requireModuleAdminAccess("placements", "services.job_postings")` walks it at runtime with explicit `__proto__` / `constructor` / `prototype` guards. Deep module, genuinely well-built; the cost is that the path is a string nothing type-checks, so a typo fails as a 403 in production.

Cross-app SSO

Two token-minting paths, both matching the §I.5.1 bridge pattern:

- **Resume bridge** (`routes/auth/bridge.ts`) — a bridge JWT with a **10-minute TTL**, signed with a **separate** `BRIDGE_JWT_SECRET`, and a redirect **host allow-list** (`polymathai.co` or a subdomain, HTTPS only) with a logged rejection otherwise. Textbook.
- **Mock-OA** (`services/oa-jwt.service.ts`) — mints an **access + refresh pair** carrying `user_id`, `tenant_id`, `org_id`, extracted into one service so both callers sign identically.

Honest gaps

Gap	Detail
Failure masking	verification errors are logged and swallowed; a forged token, an expired token, and a Secret Manager outage all produce an anonymous request. A 401 and a 503 are very different facts.
No revocation path	no token-version column, no denylist. A leaked backend JWT is valid until it expires.
The legacy twin	`req.user_id` / `req.organisation_id` / `req.tenantId` are set unconditionally alongside `req.context` — and legacy `req.tenantId` uses `user.tenant_id`, **not** the switched one. After a context switch the two disagree. Any code still reading the legacy field scopes to the wrong tenant.
Three feature-gate mechanisms	`checkFeature` (162), `requireFeature` (47), `FeatureService` (16) — see §II.2.3.
Guards aren't enforced by tooling	42 `requireRole` call sites against 586 endpoints. Nothing fails the build for an unguarded route.

The incident worth remembering

2026-05-15. A `LEARNER`-role account deleted a course through a mutating `/admin/*` route. The chain: the middleware populated context but didn't reject → the `/admin/*` prefix was assumed to be protection → most mutating routes had no role check.

Three §I.5.2 failure modes in one event — *prefix-as-security*, *unguarded new route*, and *authenticated mistaken for authorised*. Guard adoption has improved materially since (§II.1.5), but the structural fix — **CI failing on a route with no guard** — is still open, which is why this is a lesson and not yet a closed item.

II.1 Overall rating: 6.5 / 10

Solid foundations and genuinely good instincts, undermined by inconsistency and a few unfinished migrations. It works and ships — but the cognitive cost of adding a feature is still high because there's often more than one "right way."

Dimension	Score	One-line why
Architecture & layering	7	Clean service/repository layer + feature-flagged migration — but **75% of route files still call Prisma directly**.
Monorepo structure	8	Acyclic, cleanly layered workspace graph; polyglot handled sensibly. Strongest structural asset.
Data modeling	5	Overloaded tables (one row = two concerns), time/lifecycle under-modeled, hierarchy missing the cohort level.
API contract design	5	No serialization boundary (snake/camel leaks), ad-hoc validation, inconsistent envelopes, mass-assignment.
Security & authorization	6	Guards now widely applied (296 `requireAuth`, 42 `requireRole`) — up from near-zero. Contract still not enforced by tooling.
Multi-tenant discipline	6	Repository auto-scopes org where adopted — but the rule is still restated by hand in **154 files**.
Consistency / DX	4	"N ways to do X" persists — still 3 live feature-gating mechanisms.
Testing & CI	8	176 API test files, 90% coverage gate, SAST + dep scan + secret scan. Genuine strength.
Frontend	5	Good component library + design tokens — but multiple feature-flag sources and legacy components still shipped.
Observability	7	Dedicated `@repo/telemetry` OpenTelemetry package with structured logging conventions.

The through-line, unchanged: the *ideas* are right (service layer, feature flags, design tokens, CI security) but they were applied **partially**, so the codebase reads as several codebases at once. The July→August delta shows the fix is *adoption*, not redesign.

II.1.5 Status against the July review

Measured on `main`, 2026-08-11. Marked honestly — where I have no valid earlier baseline, I say so rather than invent one.

July finding	Status now	Evidence
Passthrough auth that doesn't enforce; most mutating routes lack role checks	**Improved**	296 `requireAuth()`, 42 `requireRole()`, 38 `requireModuleAdminAccess()` call sites
76% of routes call the DB directly	**Unchanged**	150 of 198 route files (75%) still call `prisma.*`

July finding	Status now	Evidence
Service layer only ~1/4 migrated	**Improving**	49 service files; 75 route files now import from `services/`; `BaseRepository` used by 8 service classes
3–5 ways to check a feature flag	**Unchanged**	still 3 live: `checkFeature` (162), `requireFeature` (47), `FeatureService`
~7,000 lines of `*-legacy.ts` kept verbatim; flags never cut over	**Partially closed**	3 `*-legacy.ts` helpers + 7 legacy dashboard components remain
Testing & CI strong	**Strengthened**	176 API test files, 90% branches/statements gate, 8 workflows
Observability	**Strengthened**	now a first-class `@repo/telemetry` package
Tenant scoping restated everywhere	**Worse in absolute terms**	`organisation_id` in 154 files / 1,330 occurrences (grew with the codebase)

Reading: authorization and testing improved materially. The two structural items — route→service migration and the "N ways to do X" consolidation — did not move. Those are the ones that need a *deadline*, not more intent.

II.2 The core engineering skills I need to strengthen

Each below: what I observed → why it hurts → the lesson → what "good" looks like → where I am.

2.1 Data modeling — model the domain's *real* dimensions as first-class

- **What I observed:** one join table carried *two* unrelated concerns (permission grants **and** unit membership) in the same row with no discriminator; "time" (semester/term) was never modeled, so per-term data has no clean home; there was no lifecycle status (active/graduated), so alumni are indistinguishable; and the org-unit tree lacked a **cohort/batch** level, so the same section unit mixed multiple admission years.
- **Why it hurts:** every consumer must defensively null-check both halves of the overloaded row; "give me batch X's semester-5 data" or "all passed-out students" becomes impossible without parsing conventions; and yearly promotion would *overwrite* membership and destroy history.
- **Lesson:** identify the domain's real dimensions — **entity, cohort, time, lifecycle, membership-over-time** — and give each its own first-class model. One table = one concern. Temporal facts need validity windows (`valid_from`/`valid_to`), not in-place updates.
- **What good looks like:** `Stream → Department → Cohort(YEAR) → Section` as the stable tree; an **append-only membership** table (transfers close a row + open a new one); a `status` lifecycle enum; a per-term **cycle** table that *all* operational data references by a single FK.
- **Where I am: Practiced** → aiming for Proficient.

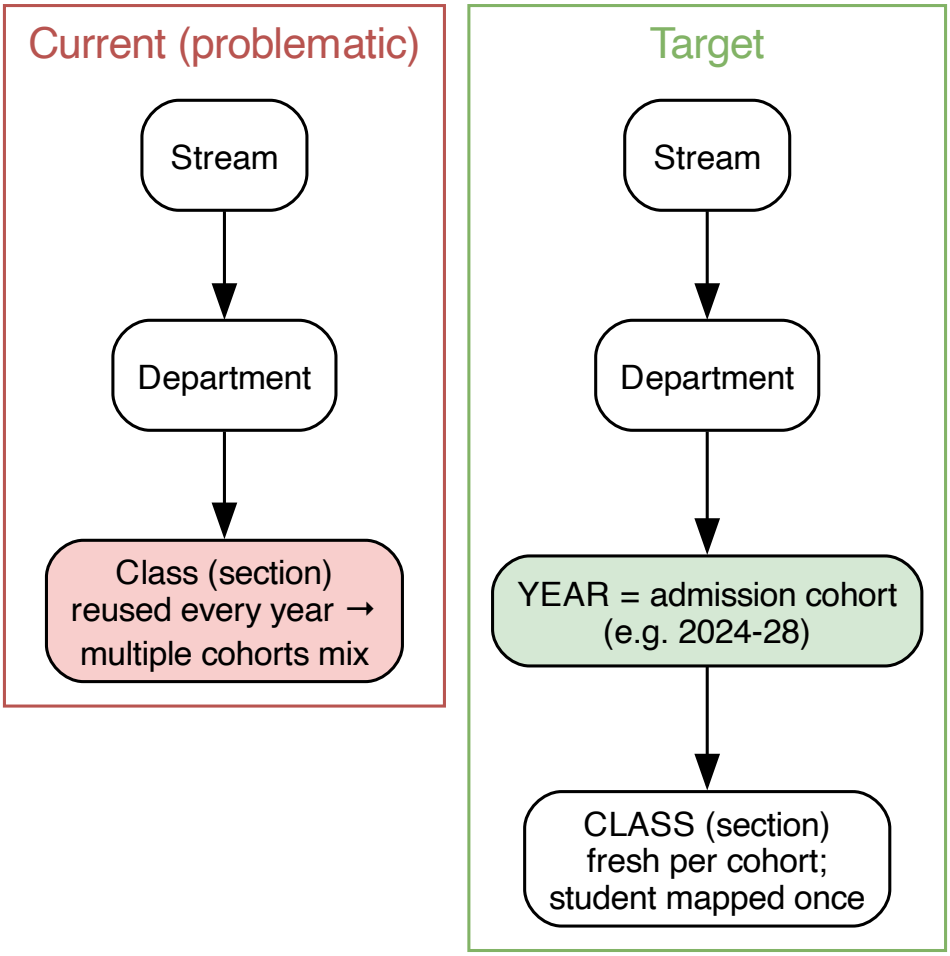

Figure 7: Current vs target unit hierarchy

2.2 API contract design — define the wire contract deliberately

- **What I observed:** no serialization boundary — DB `snake_case` leaked straight to clients in some endpoints while others used `camelCase`, sometimes **mixed in one request body**; validation was ad-hoc (`if (!x)` in most places, a schema library in one corner); write endpoints did `data: req.body` (mass-assignment); no shared pagination/filtering shape.
- **Why it hurts:** a consumer can't predict field casing; unvalidated bodies are both bugs and security holes; mass-assignment let a caller set fields (like a privileged role) they should never control.
- **Lesson:** the API contract is a deliberate design artifact. **Validate every input at the boundary**, map to an explicit **allow-list** of writable fields, and pick one casing for the wire with a single mapping layer so persistence naming never leaks.
- **What good looks like:** a schema (e.g. Zod) per endpoint that parses + types the input; one `toCamel`/`toSnake` boundary; a standard `{ data }` envelope and one pagination shape.
- **Where I am: Practiced.**

2.3 Abstraction & central components — one canonical way per concern

- **What I observed:** the **same concept implemented many times** — 3 ways to check a feature flag (still true today), 5 user-creation paths, 2 unit-creation endpoints with opposite trade-offs, 2 internal-auth schemes, multiple frontend feature-flag sources, duplicated Firebase/API-wrapper files across apps.
- **Why it hurts:** a developer can't tell which one to use, they diverge over time (one gets a fix the others don't), and each new feature adds a further variant.
- **Lesson:** the moment you write a **second** way to do something, stop and extract the first into a shared primitive. Duplication of *logic* is far more expensive than the small cost of the abstraction.
- **What good looks like:** one feature-check guard, one `createUser` service, one API client, one component library, one email helper — each imported everywhere; dead alternates deleted.
- **Where I am: Aware → Practiced.** Still my biggest gap; the metric has not moved since July.

2.4 Avoid hardcoding — if the schema is data-driven, the code must read the data

- **What I observed:** the platform advertised a *flexible, per-tenant* schema (unit types are free-form strings defined per tenant) — yet code **hardcoded** the type literals (`"CLASS"`, `"DEPARTMENT"`, `"YEAR"`) in filters and UI, and role checks compared **string literals** instead of the enum (one had a typo — `"SUPERADMIN"` — that silently locked out real super-admins). Feature keys were defined twice in two places.
- **Why it hurts:** a tenant that names things differently gets **silent empty results** (no error) — the worst kind of bug. Hardcoding negates the flexibility the schema promised.
- **Lesson:** if a value is configurable/data-driven, **read it from config/DB at runtime**; never hardcode an assumption about it. Use enums/typed constants from a single source instead of raw strings.
- **What good looks like:** resolve unit types from the tenant's schema config; a single typed `FeatureKey` union; role comparisons against the enum, never string literals.
- **Where I am: Practiced.**

2.5 Authorization & security modeling — enforce at one server-side chokepoint

- **What I observed:** the global auth middleware **populated** context but didn't **enforce** it (it called `next()` even with no token), so protection depended on each route remembering to add

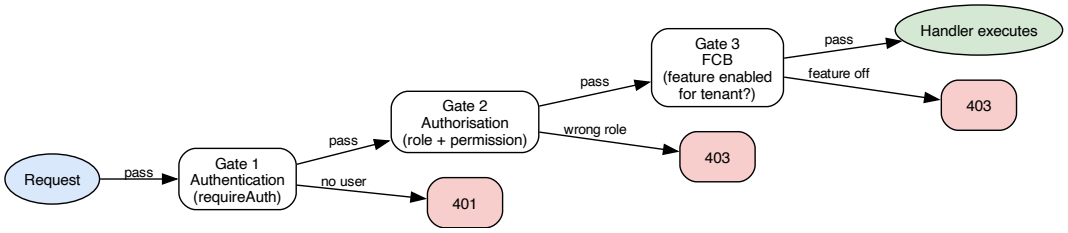

Figure 8: The three-gate request contract

guards — and most mutating routes didn't; a URL prefix (`/admin/*`) was mistaken for protection; and unvalidated role assignment allowed a tenant admin to escalate to global super-admin.

- **Why it hurts:** these are real, exploitable gaps — privilege escalation, cross-tenant access, destructive endpoints callable by low-privilege users.
- **Lesson:** authorization is **enforced on the server, at a consistent chokepoint**, never by URL naming, never by the UI. Adopt an explicit **three-gate contract — authentication → authorisation → feature** — as composed middleware on every protected route.
- **Progress:** guards are now applied broadly (296 / 42 / 38 call sites). **The remaining gap is enforcement**: nothing *fails the build* when a route ships without a guard. Convention without tooling is a wish, not a convention.
- **Where I am: Practiced → Proficient.**

2.6 Consistency & naming discipline — one term per concept

- **What I observed:** the *same* concept had many names — the tenant entity was called `tenant/entity/college/org`; the org-unit was `unit/class/section/department`; a person was `learner/student`, `instructor/teacher/professor/faculty`; CRUD verbs varied; file naming mixed four schemes.
- **The worst instance — the naming trap:** `tenant_id` means `organisation_entities.id` on *every* table — except on `organisation_entities` itself, where it is an unrelated legacy identifier. So the name is correct on every table except the one you'd consult to learn what it means. Nothing in the type system warns you. The one exception lives on the one table you'd consult to learn the rule.
- **Why it hurts:** naming drift is a tax paid on *every* future read and change.
- **Lesson:** pick **one** name per concept, then **enforce with lint**. Consistency is a feature; it's the cheapest way to lower cognitive load. And: **a name that is right everywhere except at its own definition is worse than a bad name — it is a trap.**
- **What good looks like:** a documented glossary, a fixed CRUD lexicon, `kebab-case` files, enforced by lint. Rename the legacy column to `external_tenant_ref` and delete a whole class of bug.
- **Where I am: Practiced.**

2.7 Finish migrations — a half-done migration is worse than either state

- **What I observed:** a good service-layer refactor was **feature-flagged with legacy fallbacks kept verbatim**, but the flags never got fully cut over, so both paths live on. Three `*-legacy.ts` helpers and seven legacy dashboard components are still shipped today.

- **Lesson:** migrations need a **cutover date and a deletion step**. Keeping both paths "for safety" indefinitely doubles the surface area and the cognitive load — the exact opposite of the refactor's goal.
- **Where I am: Aware → Practiced.** Still open after a month; this is the item to schedule.

2.8 Verification discipline for sensitive changes

- **Lesson reinforced:** for anything touching auth, scoping, or data integrity, write the test that proves the gate (200 / 401 / 403 / tenant-isolation) and *verify behavior*, don't assume. The strongest parts of this codebase are exactly the areas with that discipline; the weakest are the ones without.
- **Where I am: Practiced → Proficient** (176 test files and a 90% gate is real evidence).

2.9 Module depth — a simple interface over substantial functionality

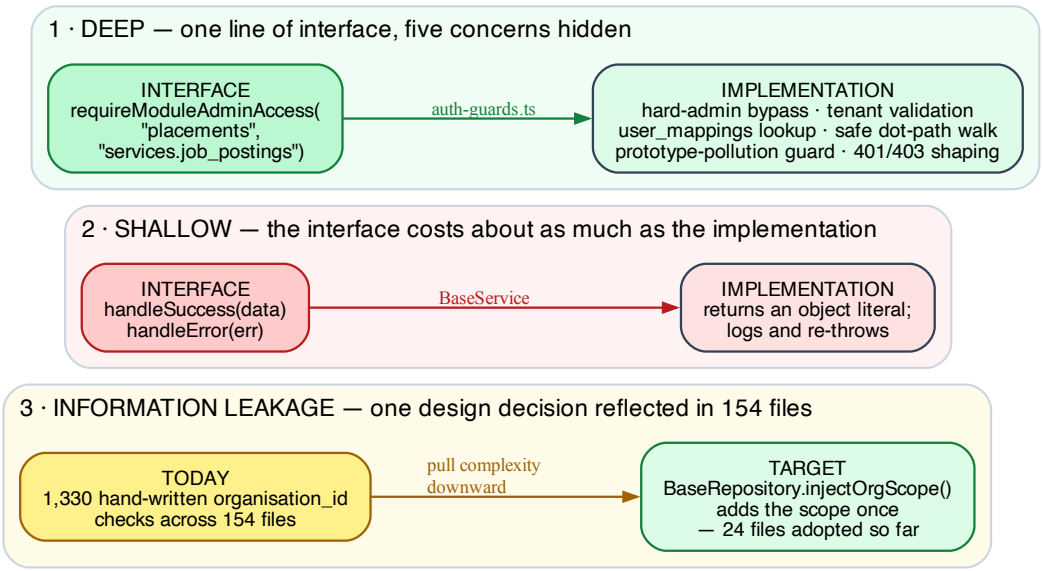

Figure 9: Module depth: deep, shallow, and leaked

- **The test (Ousterhout):** compare the cost of the *interface* against the functionality it *hides*. Deep = small interface, lots hidden. Shallow = the interface costs about as much as the implementation, so it earns nothing.
- **Deep, in this codebase:** `requireModuleAdminAccess("placements", "services.job_postings")` — one line at the call site hides hard-admin bypass, tenant validation, a `user_mappings` lookup, a safe dot-path walk through untyped JSON with prototype-pollution guards, and consistent 401/403 shaping. Also `errorHandler(ERROR_CODES.RECORD_NOT_FOUND)` — no route anywhere needs to know that maps to a 404.
- **Shallow, in this codebase:** `BaseService.handleSuccess()` returns an object literal; `handleError()` logs and re-throws. Learning to *use* the class costs more than the code it saves.

- **Lesson:** before adding an abstraction, ask what it *hides*. If the answer is "not much," it is a shallow module and it makes the system worse, not better — a new thing to learn that buys nothing.
- **Where I am: Practiced → Proficient.**

2.10 Information hiding — one decision should live in one place

- **What I observed:** multi-tenancy is a single design decision, restated by hand in **154 files** (1,330 `organisation_id` occurrences, 1,429 `tenant_id`). The fix already exists in the repo — `BaseRepository.injectOrgScope()` adds the scope once — and 8 service classes have adopted it.
- **Why it hurts:** this is the definition of **change amplification**. Adding unit-level isolation, or changing what "scoped" means, is a 154-file edit with nothing to catch the misses. It is the single largest source of complexity in the system.
- **Lesson:** when a rule must be obeyed everywhere, make it *impossible to disobey* rather than documenting it. Push it down into the layer everything already goes through — ideally a Prisma client extension, so it applies without opt-in. (This is §I.3 layer-2 enforcement applied to this codebase.)
- **What good looks like:** no route can construct an unscoped query, because the client it has cannot express one.
- **Where I am: Practiced.**

2.11 Define errors out of existence

- **Done well:** `validateTenantAccess()` returns a boolean — missing user, missing tenant, and no mapping all collapse to `false`. There is no "check failed" exception to handle separately from "access denied," because there is nothing exceptional about a user lacking access. Similarly, `responseWrapper` aggregates all error handling into one `try/catch` for 792 call sites.
- **Done badly:** an `api_logs` insert had a recovery path for "column doesn't exist yet" (Prisma `P2022`) that retried with the offending fields still present — byte-identical on exactly the fields it claimed to drop. It could never work, and the failure was swallowed.
- **Lesson:** the best exception handling is an API design where the exception cannot arise. Ask "should this be an error at all?" before writing the handler. Whether a column exists is not a runtime question — the migration either ran or it didn't; let the schema be authoritative and delete the branch.
- **Where I am: Practiced.**

II.3 Tradeoffs ledger — deliberate decisions and what they cost

Recording these matters more than recording the mistakes: a tradeoff I can name and defend is senior engineering; the same decision unnamed is an accident.

Decision	Why it was right	What it costs	Verdict
Cloud functions **outside** the pnpm workspace	They deploy standalone to GCP; isolation is a feature	Version drift — `express` at `^4.18.2` in four functions and `^4.22.1` in a fifth; `ts-jobspy` at `^1.4.0` there vs `^2.0.3` in `@repo/placements`	**Keep**, but pull `job-scouting-ingest` in — it now shares a library
Python **outside** the JS workspace	Right tool per language; pnpm cannot manage Python	`pnpm dev` doesn't start the interview service; three separate Python dependency setups (root `uv`, two `requirements.txt`)	**Accept**, document loudly; optionally add a shim `package.json`
Express over NestJS	Fastest path to shipping; minimal ceremony	Zero enforced structure — which is *why* 75% of route files still call Prisma directly. The framework never pushed back	**Accept**, but supply the discipline via lint + review
`prisma db push` over migrations	Fast iteration on a fast-changing schema	No migration folder on `main` — no replayable history, no rollback, painful new environments	**Revisit** — highest-risk item on this list
Feature-flagged refactor with **verbatim legacy fallbacks**	100% rollback safety during a risky migration	Both paths permanent until cutover; 3 `*-legacy.ts` + 7 legacy components still shipped	**Finish it** — set a cutover date
Free-form per-tenant unit types	Promised flexibility across institution shapes	Code hardcodes the literals anyway, so it pays the complexity without the benefit	**Fix** — bind to semantic level-roles (§II.8.2)
Turbo **local cache**, unbounded	Fast rebuilds	100 GB in `.turbo/cache` (488 artifacts, largest 7.7 GB) on a 95%-full disk	**Prune**, and investigate why artifacts are GB-scale
No `test` task in `turbo.json`	Nothing forced the decision	`turbo test` does not exist; tests only run per-app and in CI	**Fix** — one-line change

---

II.4 What "good" looks like — the target so a new engineer onboards easily

The fastest way to lower onboarding cost is to make the codebase **predictable** — one obvious way to do each thing:

1. **One request contract**: validate at the boundary, allow-list writes, camelCase wire, one envelope, one pagination shape.
2. **One security chokepoint**: `guard({ roles, feature })` on every protected route (the three-gate contract); backend enforces, frontend hints — and **CI fails** if a route registers without one.
3. **One way per concern**: one feature-check, one `createUser`, one unit-create, one API client, one component library. Delete the alternates.
4. **A glossary + lint**: one canonical term per concept, one file/CRUD convention, enforced automatically.
5. **First-class domain models**: entity, cohort, time (cycle), lifecycle status, append-only membership — no overloaded rows, no hardcoded type/role strings.
6. **Scoping that cannot be forgotten**: tenancy enforced in the data layer, not restated in 154 files.
7. **Finish what you start**: flags get cut over and legacy code deleted on a schedule.

A new engineer should be able to read one page of conventions and correctly guess how any feature is built.

II.5 Incremental improvement roadmap

Ordered by complexity removed per hour spent — each step stands alone.

1. **Restore migrations.** Baseline the current schema as an initial migration and stop using `db push` for anything shipped. Highest risk, cheapest fix.
2. **Add the `test` task to `turbo.json`** (`{"dependsOn": ["^build"]}`) so `turbo test` covers the repo.
3. **Make scoping impossible to forget** — move `injectOrgScope` into a Prisma client extension so it applies without opt-in. Attacks the 154-file leak at the root with no route rewrite.
4. **Lint the guard contract** — fail CI when a route registers without an auth guard. Converts a convention into a rule.
5. **Pick one feature gate, delete two.** Keep `checkFeature` (162 sites, actually works); make `req.context.features` carry real flags or drop `requireFeature`.
6. **Set a cutover date for the legacy paths** and delete the 3 `*-legacy.ts` helpers + 7 legacy components.
7. **Continue route → service migration**, one route per feature touched, not as a refactor sprint. Track the 75% number monthly.
8. **Fix the data model** where multi-year matters: cohort level, cycle table, lifecycle status, membership split.
9. **Rename `organisation_entities.tenant_id`** to `external_tenant_ref`.

II.6 Meta-lessons

6.1 The second implementation is the signal

When I notice myself writing "the second way to do X," that's the signal to build the shared primitive instead. Nearly every problem here — the 3 feature checks, the 5 user-creation paths, the naming drift — is the same root cause: local decisions made without a shared convention. Good senior engineering is disciplined consistency, not clever one-offs.

6.2 Conventions without tooling are wishes

The pattern repeats: a good abstraction is written, documented as *the* way, and then not adopted — because nothing forced it. `requireRole` sat at zero call sites for months while it was documented as the standard. `BaseRepository` was written, documented, and imported by nothing. The lesson is not "write better docs." It is: **if it matters, make CI fail without it.**

6.3 Verify what you are looking at before you conclude anything

On 2026-08-11 I ran a full architectural review against a feature branch that was **181 commits and four months behind `main`**. It concluded there were 149 models (there were 199), zero `requireRole` call sites (there were 42), no telemetry package (there was one), and a 3,523-line route file that had already been split. Nearly every "finding" was work the team had already completed.

The habit: before reviewing, benchmarking, or reporting on any codebase — mine or someone else's — establish *which commit* you are reading.

```
git log --oneline -1 main
git rev-list --left-right --count main...HEAD
```

This generalizes past git: stale dashboards, cached query results, a local `.env` pointing at the wrong database. **A confident conclusion drawn from the wrong snapshot is more damaging than no conclusion**, because it gets written down and acted on. Cheap verification first, confident claims second.

II.7 My growth plan & working method

The shift I'm making: from breadth to depth. I've shown I can build wide and ship to scale; the next level is engineering *discipline* — building systems the next ten engineers can extend without pain. Concretely, three practices:

1. **Understand what I built and *why* — make the implicit explicit.** For each subsystem, write down *what* it does, *why* I built it that way, and *what tradeoff* I was (or wasn't) consciously making. This converts tacit knowledge into transferable understanding, and surfaces which decisions were **accidental** versus **deliberate**. The accidental ones are exactly the ones worth redesigning. §II.3 above is the first pass at this.
2. **Redesign best-practices-first — don't pattern-match the past.** When redesigning an API or component, start from the **ideal**: what's the right contract, the right boundary, the one obvious way? **Decide the right way before writing code**, then implement to that standard. The existing code is *input*, not the template.
3. **Clean incrementally — one convention at a time** (per §II.5). Establish the convention → write it down → **enforce it in CI** → migrate to it. Not a big-bang rewrite.

How I'll know the discipline is landing: - I can explain the *why* behind every significant design choice — and honestly say where "I'd do it differently now." - There's **one obvious way** to do each common thing, and it's written down and enforced. - The tracked metrics in §II.1.5 move in the right direction month over month.

The mindset: treat this cleanup not as "fixing mistakes," but as **upgrading past-me's decisions with present-me's judgment — and writing the reasoning down so future-me and the next engineer inherit the judgment, not just the code.**

II.8 Two design principles this review reinforced

8.1 Model the access patterns *before* the schema

Before designing a model, **imagine how the data will actually be read and written** — the queries, the filters, the scoping dimensions, who asks for what — and shape the schema to serve those cheaply. The schema is the *implementation* of the access patterns, not something designed in the abstract and queried around afterward.

The batch case is the proof: in a college the dominant read is **"scope everything by cohort/batch."** So batch **has to be a first-class dimension from day one** — retrofitting it (roll-number inference, denormalized batch columns, awkward joins) is exactly the pain I hit. If I'd written the top 10 queries first, "filter by batch" would have been #1, and the missing cohort level would have been obvious before a line of schema was written.

> This matters in relational, but it's *non-negotiable* in denormalized/NoSQL stores where you can't just add an index or join later — there, the access pattern **is** the schema.

8.2 Flexibility must earn its complexity

The per-tenant free-form schema (arbitrary unit `type` strings, per-tenant schemas, metadata blobs, adjacency-list tree) is powerful **only if two things hold**:

1. I genuinely have tenants with **structurally different** hierarchies — not the *same* shape with different names/depths.
2. The code is disciplined enough to be **fully data-driven** — it never hardcodes a type/level name.

Today the codebase fails #2, so it pays the full **cost** of flexibility — no type safety, recursive tree queries, implicit metadata schemas, high cognitive load — **without the benefit**: a differently-named tenant silently breaks. That's *accidental complexity I'm not cashing in on.*

The resolution depends on the honest answer to "how different are my tenants, really?":

- **Same shape, different labels** → use a **fixed, well-named schema with configurable labels/depth**. ~90% of the benefit, a fraction of the complexity.
- **Genuinely different structures** → keep flexibility, but bind it to **semantic level-roles** (`root`, `grouping`, `enrollment_unit`, `leaf`) that tenants map their named levels onto, so code reasons about *roles* — "the enrollment level" — never literal names. Hardcoding then becomes impossible.

The lesson: adopt a generic/flexible design only when the variation is real **and** the team will pay the discipline tax to keep the code data-driven. Flexibility the code doesn't honor is just complexity.

What I'd do differently, in order (the pooled-model summary)

1. Enforce scoping at layer 2 from day one (client extension), never at call sites.
2. Model the hierarchy's real levels — including cohort/batch and time — before writing the schema.
3. One canonical name per concept; rename the legacy column immediately.
4. Write case-4 isolation tests alongside the first endpoint, not later.
5. Keep tenant→datasource resolution behind an interface, so hybrid stays cheap if an enterprise contract ever demands a silo.